

Obesity Estimation based on eating habits and physical condition

María Colomer, Sara Díez & Gússem Yahia-Cheikh

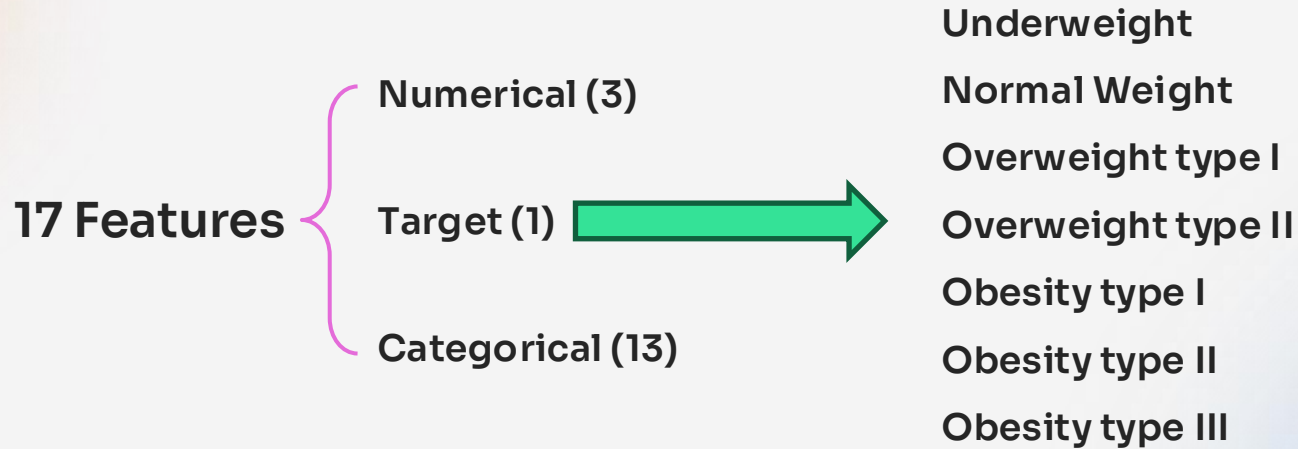
Goals

1. **Develop multiple supervised classifiers for obesity type prediction**
 2. **Compare model behavior, interpretability, and sensitivity to data structure**
 3. **Evaluate individual model performance on a shared dataset**
 4. **Implement ensemble learning using hard and soft voting strategies**
 5. **Demonstrate performance gains through model aggregation**
-

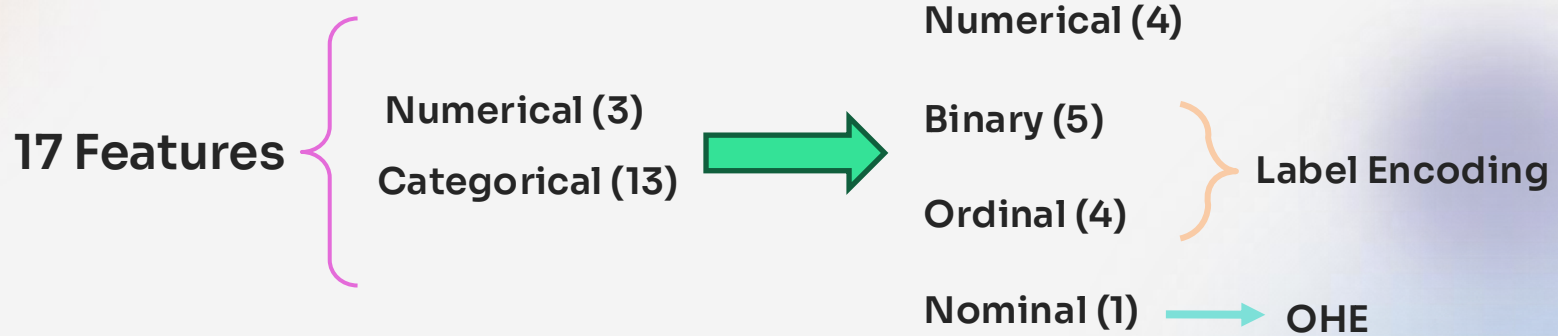
Dataset Overview & Preprocessing



Dataset Overview & Preprocessing



Dataset Overview & Preprocessing



Exploratory Data Analysis (EDA)

Goals

- Understand the dataset.
- Evaluate the dataset quality.
- Find possible issues.
- Identify patterns.

...What about outliers?



Target variable Plot

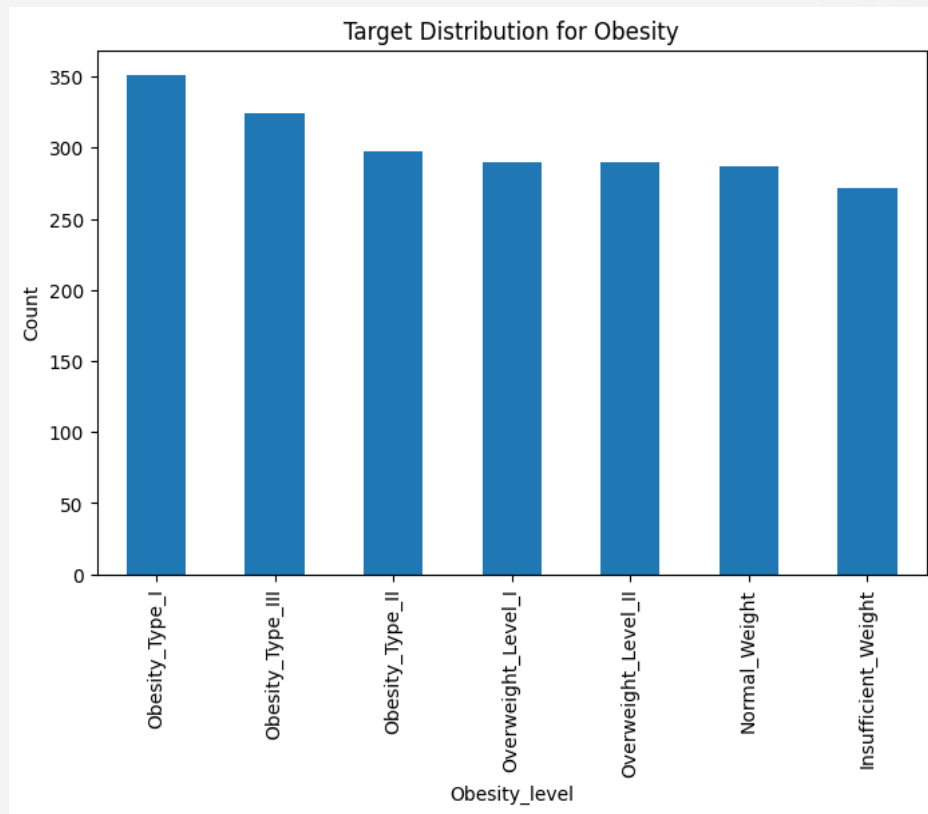
Boxplot Analysis

Correlation Matrix

Target Distribution Plot

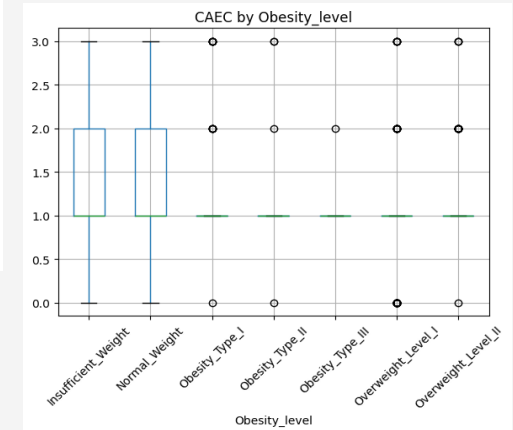
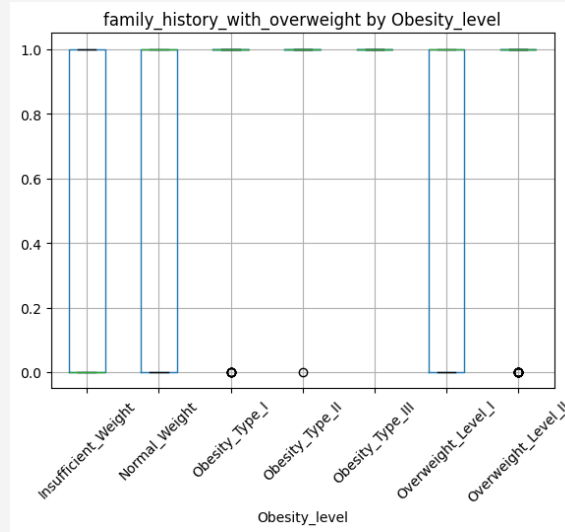
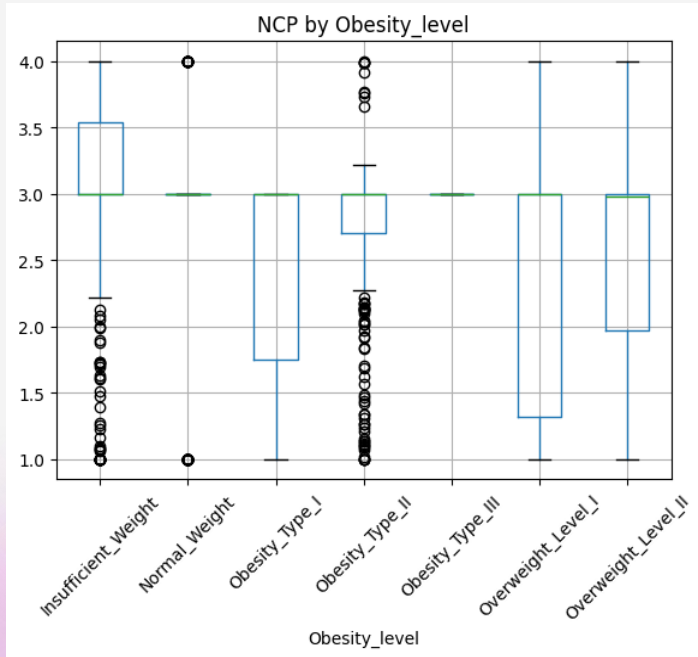
Aim: visualize the distribution of the target variable.

- Unbalanced distributions.
- Categorical and multiclass character.
- Bias toward majority classes.
- Issues in modelling and evaluation metrics.



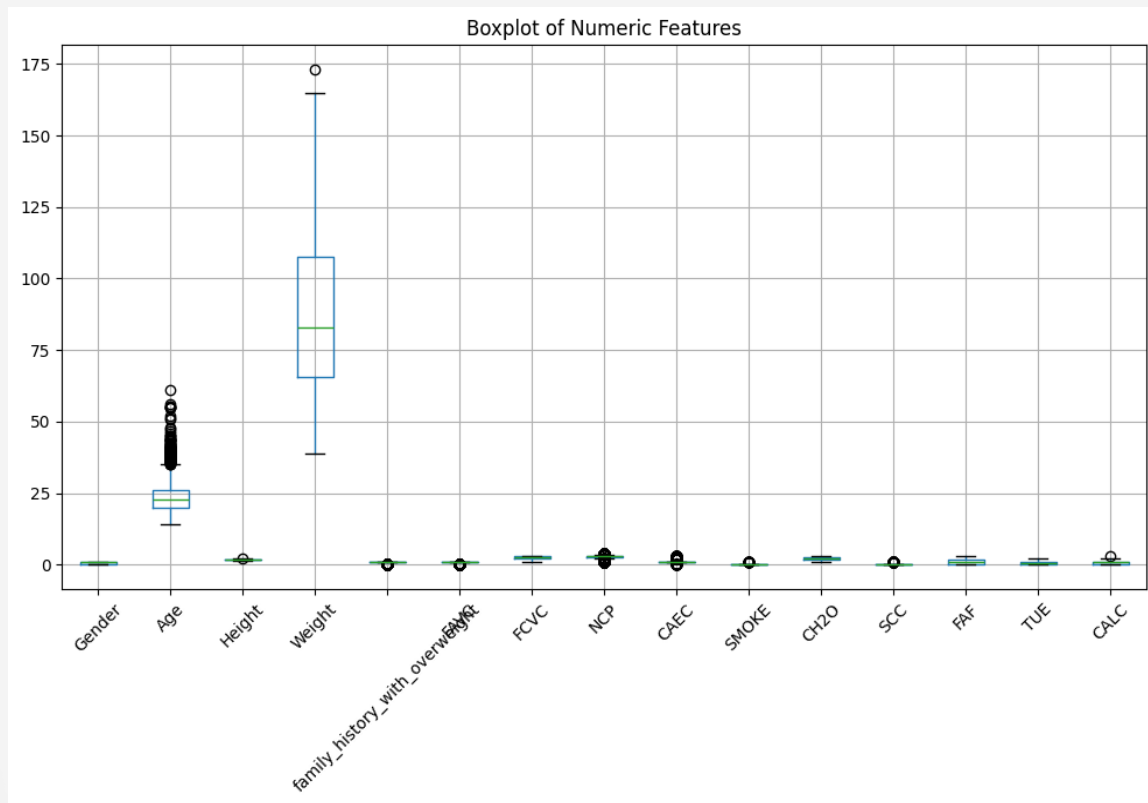
Boxplot by obesity level

Aim: compare numeric features with target variable.



Boxplot for all numeric features

Aim: interpret & inspect spread, skewness & outliers.



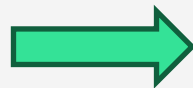
Boxplot by obesity level, for all numeric features

Key Interpretations

Identification of relevant features.

Appearance of extreme values.

+ Results Target Distribution Analysis



**Must handle outliers
for a better result.**

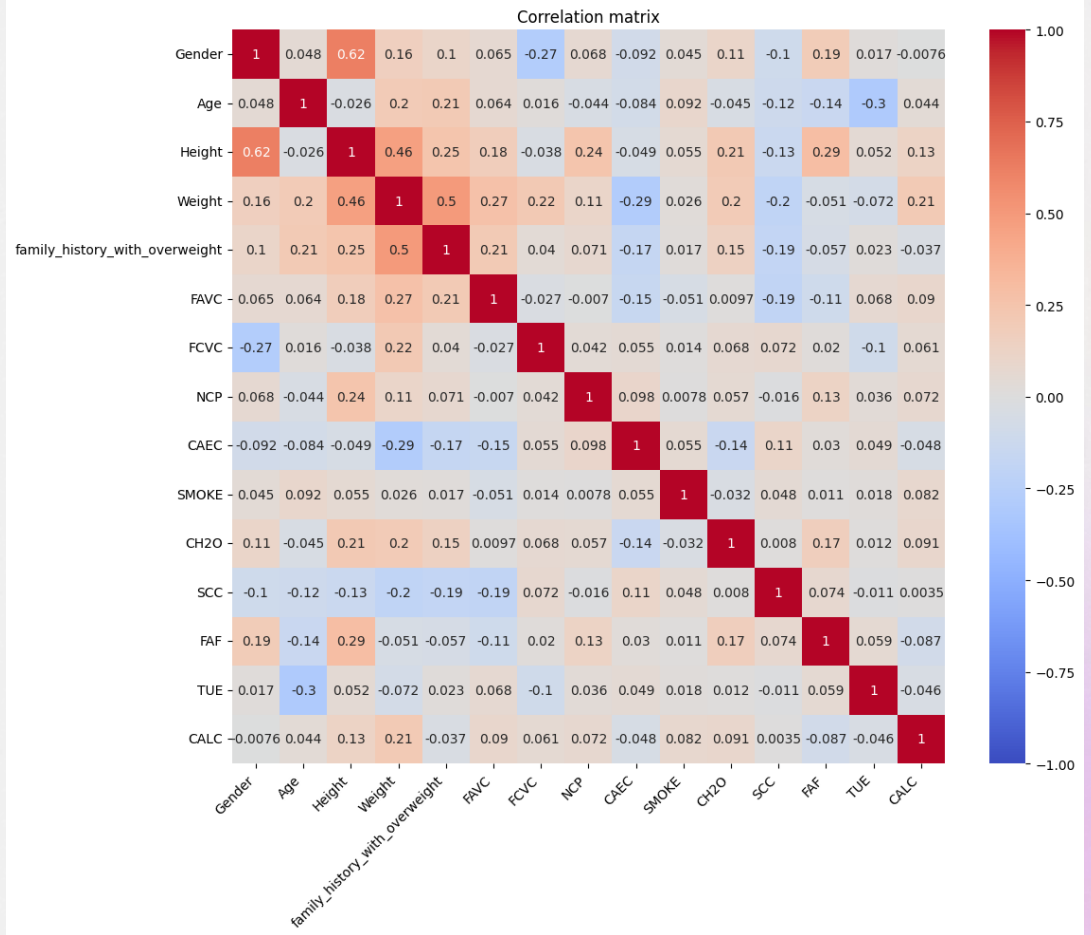
Correlation

Aim: assess relationships between features.

Features highly correlated.

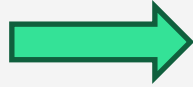
Correlation with obesity metrics.

Redundancy.



Detection & Handling Outliers

Step 1: Detection
IQR METHOD



Total number of outlier values (outliers per column): 1866.
Total number of rows that contain at least one outlier: 1131.

Step 3: Handling
IQR CAPPING

Modelling

Tree-Based Models

Statistical and Probabilistic Models

Margin-Based Model

Ensemble Models

Tree-Based Models

Decision Tree *with Hyperparameter Tuning*

Step 1: Dictionary of parameters:

```
{'max_leaf_nodes': list(range(2, 300)), 'min_samples_split': list(range(2, 20))}
```

Step 2: Grid Search of the Decision Tree Classifier using 5-fold cross validation

Step 3: Training & insights → depth: 14 , n_leaves: 217

Step 4: Evaluation → accuracy = 0.773

Statistical and Probabilistic Models

Logistic Regression Pipeline

Step 1: Preprocessing Pipeline (Numerical & categorical Features, ColumnTransformer())

Step 2: Preprocessing + Logistic Regression = LOGISTIC REGRESSION PIPELINE

Step 3: Training

Step 4: Evaluation

Accuracy = 55.68 %

ROC AUC (OvR) = 87.46 %

Statistical and Probabilistic Models

Naive Bayes

Step 1: Preprocessing Pipeline (`Standard Scalerer()` & `GaussianNB()`)

Step 2: Training & predict

Step 3: Training & evaluation → **accuracy: 0.3996**

confusion matrix:

```
[[55  0  0  6  2  0  5]
 [36  2  0 23  3  0  8]
 [21  1  0 42  0  0 24]
 [ 2  1  0 62  0  0  9]
 [ 0  0  0  0 81  0  0]
 [29  1  0 31  0  0 12]
 [20  1  0 40  0  0 11]]
```

Margin-Based Model

Support Vector Machines

1. **Scale the data with `StandardScaler()`**
2. **Set parameter `C` to control the trade-off between margin maximization and classification error.**
3. **Set non-linear kernel (RBF)**

Margin-Based Model

Support Vector Machines

1. Scale the data with `StandardScaler()`

2. Set parameter `C` .



72% Accuracy

3. Set non-linear kernel (RBF)

Ensemble Models

Random Forest

Step 1: RandomForestClassifier() parameter tuning:

`(n_estimators=200, bootstrap=True, n_jobs=-1, random_state=RANDOM_SEED)`

Step 2: Training and evaluation → accuracy: 0.8806

Step 3: Feature Importance evaluation

	Feature	Importance
1	Age	0.166131
2	Height	0.154155
5	FCVC	0.134189
12	TUE	0.103949
11	FAF	0.102338

Ensemble Models

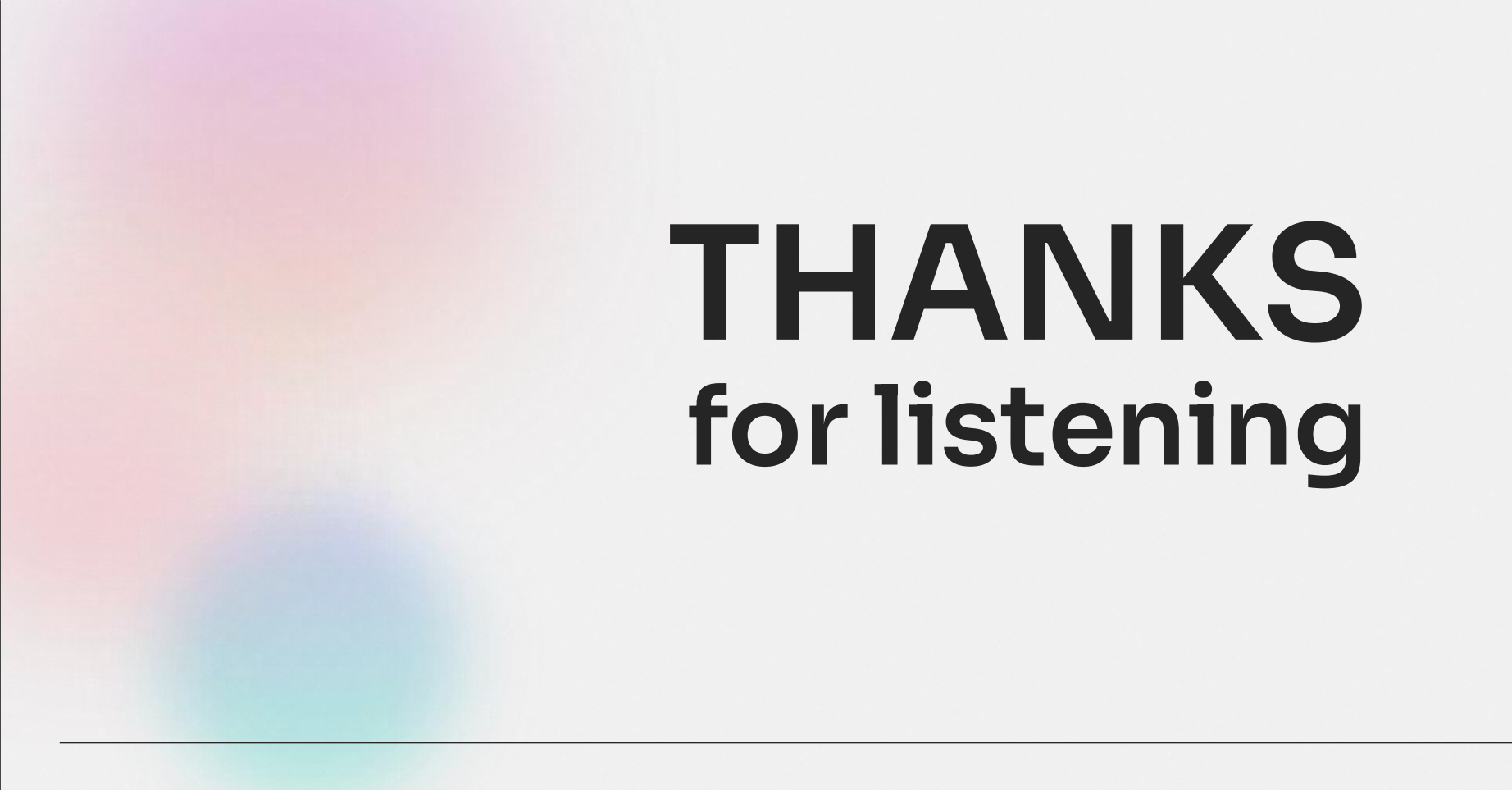
Voting Classifier

Step 1: Combining multiple classifiers (`LogisticRegression()`, `GaussianNB()`, `SVC()` & `RandomForestClassifier()`)

Step 2: Training `VotingClassifier()` **with parameters** `voting='soft'` & `voting='hard'`

Step 3: Compare Classifier accuracies:

- LogisticRegression: 0.5095
- GaussianNB: 0.3996
- SVC: 0.7330
- RandomForestClassifier: 0.8826
- VotingClassifier (Hard): 0.7235
- VotingClassifier (Soft): 0.7292



THANKS
for listening